

Vibe2Spec

面向 Codex 的 Spec-first 软件开发流程

大模型驱动的软件开课程项目报告

学生姓名： 史志远

学号： 2022011283

项目类型： 本地 Codex CLI 与软件流程实验

核心主题： Codex、本地开发流程、需求工程、验证

主要产物： CLI harness、Codex skill、artifact 校验、对比实验

报告日期： 2026 年 6 月

源码仓库： github.com/Dynamite2003/codex-agent-harness

本报告总结 Vibe2Spec 项目的问题背景、流程设计、系统实现和实验结果。项目目标不是重新实现一个编码模型，而是在 Codex 外层增加一组可落盘、可检查、可复盘的工程约束：先把需求、设计、任务和实现 prompt 写成文件，再进入编码和验证。

摘要

本项目围绕 Codex 本地开发流程做了一层 Spec-first 包装。具体做法是把一次开发目标拆成需求、设计、任务和实现 prompt 四类 artifact，再由 CLI harness 或 Codex skill 推进后续实现。完整 harness 会记录 prompt、上下文、命令、输出和状态，适合课程展示、失败复盘和过程审计；轻量 skill 直接在当前 Codex 会话中生成同类文档，适合日常开发。三组实验结果为：Todo deadline 任务中 Direct baseline 与 Vibe2Spec 均通过 16/16 功能探针；Sprint Board Lite 中二者核心功能接近，但 Vibe2Spec 保留了更完整的过程文件；订阅 proration 计费任务中 Direct baseline 通过 2/6，Vibe2Spec 通过 6/6。实验说明，这套流程的主要作用是把隐含规则提前写成需求、设计决策和验收 fixture，而不是增加代码生成步骤。

关键词：Codex；Spec-first；需求工程；验证；开发流程

1 项目背景与问题定义

Codex 已经可以在本地仓库中阅读代码、修改文件并运行测试。对小范围修改，直接对话通常足够；但当任务包含隐含业务规则、跨模块影响或后续维护要求时，问题往往出现在编码之前：需求在聊天中被自由解释，关键设计决策缺少记录，测试只覆盖表层行为，失败时也缺少可审计状态。

本项目关注的是编码前的约束如何落地。在 proration 计费实验中，原始需求只说“按天计费”，Direct baseline 将账期固定为 30 天，忽略 2 月、闰年和 31 天账期。这个问题不是语法错误，而是业务边界没有进入需求和验收标准。

Vibe2Spec 的处理方式是在 Codex 外层增加文件化流程，先生成可审阅 artifact，再进入实现阶段。项目不试图替代 Codex 的编码能力，而是把“让 Codex 做什么、可以看什么、必须产出什么、失败时如何停止”显式写入配置和运行记录中。

1.1 研究问题与项目贡献

本项目围绕三个问题展开：第一，如何把一句自然语言开发需求转化为可审阅、可实现、可验证的 artifact 链；第二，如何在不强制多 agent、不显著拖慢日常开发的情况下，把 Spec-first 流程内化到 Codex 使用体验中；第三，在哪些任务上，Vibe2Spec 相比直接 Codex 开发能体现实际收益。

围绕上述问题，项目形成了以下可运行产物：

- 实现 requirements -> design -> tasks -> implementation 四阶段 CLI harness；
- 将阶段输入、prompt、上下文清单、执行命令、stdout/stderr、状态文件和 manifest 落盘；
- 提供 vibe2spec-flow skill，让日常使用默认不启动完整 harness；
- 引入 artifact 内容校验，覆盖 Product Archetype、Success Mode、Failure Modes、Behavioral Requirements、Quality Requirements 等质量镜头；
- 默认实现阶段改为单 agent 顺序执行，subagents 只作为上下文过大或任务独立时的可选策略；
- 通过 Todo、Sprint Board、Proration 三组实验区分“小任务打平”“复杂 UI 流程有过程收益”“隐含业务边界有功能收益”三类情况。

2 方法设计

2.1 四阶段 Artifact 链路

Vibe2Spec 固定采用 requirements、design、tasks、implementation 四个阶段。阶段之间只通过落盘 artifact 传递上下文：设计阶段读取需求文档，任务阶段读取需求和设计，实现阶段读取需求、设计和任务。表 1 给出主要产物和作用。

这种 artifact-only 模式并不追求复杂平台化，而是把一次自然语言开发目标变成一组稳定文件。每个阶段都能回答四个问题：当前只做什么、可以读取什么、必须产出什么、怎样判断完成。

Table 1 Vibe2Spec 四阶段 artifact 链路

阶段	产物	作用
需求	doc/proposal.md	明确背景、目标、非目标、用户、失败模式、功能需求、质量需求、验收标准和验证策略。
设计	doc/detailed-design.md	明确模块职责、数据模型、状态流、接口契约、ADR、风险响应和测试策略。
任务	doc/tasks/	将设计拆成可执行 checklist，记录文件范围、依赖关系、追溯关系和进度。
实现	doc/prompt.md	生成实现 prompt，要求顺序编码、验证、回写任务进度并记录偏离。

2.2 Artifact-only 上下文隔离

默认配置启用 `new_conversation_per_phase` 和 `artifact_only_context`。也就是说，每个阶段都应该是新的 Codex 对话，阶段之间不依赖聊天历史，只通过配置声明的 artifact 传递上下文。例如，设计阶段读取 `doc/proposal.md`，任务阶段读取 `doc/proposal.md` 和 `doc/detailed-design.md`，实现阶段读取需求、设计和 `doc/tasks/`。运行时还会生成 `context.json`，记录本阶段声明了哪些上下文输入、哪些实际存在、哪些缺失，以及预期输出是什么。

2.3 Harness 与 Skill 两种入口

项目提供两条入口。CLI harness 会在目标项目中生成 `.harness/runs/<run-id>/`。每个阶段目录包含 `prompt.md`、`context.json`、`command.sh`、`stdout.txt`、`stderr.txt`、`status.json` 和 `conversation/`；运行根目录包含 `manifest.json`，记录用户目标、项目路径、阶段文件和隔离策略。这一路径主要用于课程展示、失败复盘、CI 风格复跑和需要 `stdout/stderr` 的场景。

CLI harness 的成本是等待更长、交互更重。日常使用时可以改用 `vibe2spec-flow skill`：它不启动四阶段 CLI，而是在当前 Codex 会话中维护需求、设计、任务和实现 prompt 四类文档。两条路径使用同一套文档结构，只是运行成本不同。

2.4 运行控制与交互协议

执行模式下，每个阶段可抽象为小状态机。准备阶段先写入 `prompt`、`context` 和 `command`；命令返回非零时，状态记为 `command failed`；输出独占行 `HARNESS_NEEDS_USER_INPUT` 时，状态记为 `needs input`；expected outputs 缺失时，状态记为 `missing outputs`；产物存在但内容校验失败时，状态记为 `artifact invalid`；只有命令成功、产物存在且校验通过时，阶段才进入 `ok`。

早期实现曾根据“请确认”“需要补充”等自然语言关键词判断 Codex 是否需要用户输入。实验显示这会误触发，因为这些词可能出现在 `prompt` 模板、日志或普通总结中。当前版本改为显式协议：只有输出中存在独占一行的 `HARNESS_NEEDS_USER_INPUT` 时，harness 才暂停当前阶段，并把后续问题写入 `needs-user-input.md`。

项目没有默认使用 `subagents`。小任务中并行 agent 会增加文件所有权、合并顺序和上下文冲突成本，因此默认实现阶段改为单 agent 顺序执行；只有任务彼此独立、文件范围清晰且上下文明显过长时，才允许可选启用子 agents。

2.5 运行目录与审计文件

每次完整运行会在目标项目下创建 `.harness/runs/<run-id>/`。每个阶段目录包含：

- `prompt.md`：本阶段传给 Codex 的 prompt；
- `context.json`：本阶段允许使用的上下文和预期产物；

- `command.sh`: 渲染后的 Codex CLI 命令;
- `stdout.txt` / `stderr.txt`: 执行模式下保存 Codex 输出;
- `status.json`: 阶段执行状态;
- `needs-user-input.md`: Codex 按协议要求用户补充信息时记录问题。

运行根目录还会生成 `manifest.json`, 记录 `run id`、项目名、项目根目录、用户目标、隔离策略和所有阶段文件路径。这样在阶段失败时, 可以区分是 Codex 命令失败、用户信息不足、产物缺失, 还是 artifact 内容不合格。

3 系统实现

3.1 核心模块

核心代码位于 `src/codex_harness/`。表 2 列出主要模块职责。

Table 2 核心模块职责

模块	职责
<code>cli.py</code>	CLI 参数解析、子命令分发、交互输入读取、默认配置路径处理。
<code>config.py</code> <code>prompting.py</code>	JSON 配置解析、字段校验、阶段顺序约束和数据结构定义。 渲染四段式 prompt, 注入上下文清单、全局约束和隔离策略。
<code>runner.py</code>	创建 <code>run</code> 目录、执行命令、记录状态、处理用户补答和缺失产物。
<code>artifact_validation.py</code> <code>skill_flow.py</code> <code>python_bootstrap.py</code>	检查 Spec-first artifact 的必要结构和质量镜头。 安装 <code>bundled skills</code> 并初始化 <code>skill</code> 工作流目录。 生成 Python 项目的 <code>uv</code> 、 <code>ruff</code> 、 <code>mypy</code> 、 <code>pytest</code> 配置。

CLI 能力覆盖运行、初始化、artifact 校验、Python bootstrap 和旧 `run` 清理。其中 `start` 是默认入口; 如果用户直接运行 `./harness -C <path> "goal"`, 程序会自动规范化为 `start`, 降低课程演示和日常使用门槛。

3.2 CLI 能力与执行行为

当前 CLI 支持 `init-skill-flow`、`validate-artifacts`、`init`、`run`、`start`、`bootstrap-python` 和 `clean-runs`。其中 `run --execute` 会真实调用 Codex CLI, 并按阶段检查目标项目中的产物; `--dry-run` 只生成 `prompt`、`context`、`command` 和 `manifest`, 不调用 Codex, 也不生成业务 artifact。

执行模式还支持几类恢复动作: 用户补答会写入 `user-answers.md` 并追加到当前阶段 `prompt` 后重跑; 用户认为当前阶段产物已足够时可用 `NEXT_PHASE` 强制进入下一阶段; 阶段结束后缺少产物时, `harness` 会提示补充重试指令, 也允许用 `SKIP_PHASE` 记录审计后跳过。命令失败、用户追问、强制进入下一阶段和缺失产物跳过都会写入状态或审计文件。

3.3 配置、Prompt 与 Artifact 校验

配置文件要求包含 `project_name`、`workspace`、`codex.command` 和 `phases`, 阶段顺序必须严格等于 `requirements -> design -> tasks -> implementation`。固定阶段顺序牺牲一部分通用工作流灵活性, 但换来更稳定的工程约束和更简单的测试边界。

每个阶段 `prompt` 固定为“目标、输入、输出、步骤”四段。渲染时注入用户目标、项目根目录、`run` 目录、阶段目录、上下文文件、全局约束和对话隔离策略。默认实现 `prompt` 会先要求 agent 识

别项目技术栈和可用工具，再选择验证方式，避免把 Python 项目的 uv、pytest、mypy、ruff 约束强加给静态 Web 或无依赖项目。

Artifact 校验被改为规则表驱动。以 doc/proposal.md 为例，校验要求覆盖 Context、Goals & Non-Goals、Target Users、Product Archetype / Success Mode / Domain Lenses、Failure Mode Lens、Functional Requirements、Behavioral Requirements、Quality Requirements、ADR Candidates、Acceptance Criteria、Verification Strategy 和 Risks。该校验仍是启发式检查，不能替代人工评审，但能防止明显的结构缺失。

3.4 已实现能力小结

截至本报告整理时，项目已经具备一个可运行的 0.1.0 版本：可以通过 ./harness、harness 或 codex-harness 启动；可以自动查找目标项目中的 harness.json，没有时生成默认配置；可以 dry-run 生成完整四阶段 prompt、上下文文件、命令脚本和 manifest；可以 execute 真实调用 Codex CLI，并把 prompt 通过 stdin 传给 Codex；可以检查 expected outputs、处理显式追问、接收用户补答、处理缺失产物、记录命令失败，并为 Python 项目生成基础质量工具配置。

4 默认流程演进

4.1 从功能清单到质量镜头

早期默认需求文档更接近功能清单，包含 Context、Goals、User Stories、Functional Requirements、ADR Candidates 和 Acceptance Criteria。实验显示，这对简单任务足够，但对 proration 这类隐含业务规则任务不够。模型可能实现看起来合理的功能，却遗漏真实账期天数、闰年、退款符号和最终四舍五入。

因此，默认配置要求需求阶段识别 Product Archetype、Success Mode、Domain Lenses 和 Failure Modes。Success Mode 包括 Correctness、Usability、Experience、Reliability、Throughput、Exploration 和 Compliance/Safety。Failure Modes 包括结果错误、流程混乱、边界状态缺失、数据丢失、权限泄漏、失败后无法恢复、AI 幻觉或越权等。目的不是让文档变长，而是让需求阶段先回答“这个项目最可能错在哪里”。

4.2 单 Agent 默认策略与技术栈适配

早期实验中，实现 prompt 倾向于鼓励 subagents。但对课程项目和小型仓库，子 agent 并行会增加文件所有权、合并顺序和上下文冲突成本。当前默认实现阶段改为单 agent 顺序执行，只有当任务彼此独立、文件范围清晰且上下文明显过长时，才允许可选启用子 agents。

另一个实验问题是默认 prompt 过度绑定 Python 工具链。对于静态 Web 项目，强制要求 uv、pytest、mypy、ruff 会引入无关环境问题。当前默认实现 prompt 要求 agent 先识别项目技术栈和可用工具，再选择验证方式。Python 项目优先沿用已有测试命令；只有已经 bootstrap 或已有相应配置时，才使用 uv、pytest、mypy 和 ruff。

5 实验设计与结果

实验不试图证明 Vibe2Spec 在所有任务上都优于 Direct Codex。更合理的问题是：在哪些任务上，Spec-first 的额外成本能换来更高正确性或可追踪性？因此项目设置三类任务：简单功能任务 Todo deadline、中等复杂 UI 任务 Sprint Board Lite、隐含业务规则任务 Proration 计费边界。

5.1 Todo Deadline 与 Sprint Board Lite

Todo deadline 任务是给无依赖 Todo Web App 增加截止时间和逾期标记。Direct baseline 与 Vibe2Spec 都通过 16/16 功能探针。该实验说明：需求清楚、反馈快的小任务不一定需要完整流程，Vibe2Spec 在这里主要增加过程记录。

Table 3 三组实验结果摘要

任务	Direct baseline	Vibe2Spec	主要结论
Todo deadline	16/16 功能探针	16/16 功能探针	简单任务打平，Direct 已足够高效。
Sprint Board Lite	核心契约通过	核心契约通过， artifact 链路完整	功能接近，过程文件更多。
Proration 计费	2/6 探针通过	6/6 探针通过	账期边界被 fixture 覆盖。

Sprint Board Lite 任务是实现一个无依赖静态 Web 看板，包含任务新增、四列状态流转、筛选、KPI、本地持久化和 JSON 导入导出。两组都通过核心契约测试，但 Vibe2Spec 保留了 proposal、detailed-design、tasks、prompt 和 verification 链路。该实验还暴露了 harness 本身问题：追问检测过宽、实现 prompt 过度绑定 Python 工具链、subagents 默认策略过重。这些问题后来被修复，并被纳入回归测试。

Table 4 Sprint Board Lite 对比摘要

维度	Direct Codex	Harness Flow
功能完整性	契约测试通过	契约测试通过，并补充 focused tests 和 DOM smoke
桌面端结构	可用，但 JSON 区留白和截断明显	表单、筛选、导入导出和看板层级更清楚
移动端可追踪性	存在裁切问题 缺少需求/设计/任务链路	仍有裁切，未完全合格 proposal、design、tasks、prompt、verification 全链路可追溯
流程成本	快、简单	更慢，暴露出 harness 误触发和环境绑定问题

5.2 Proration 计费边界

Proration 实验用于检查隐含业务规则。原始需求只有一句话：做一个订阅升级/降级的按天计费计算器，输入当前月费、新月费、账期开始/结束日期、变更日期、优惠券和税率，输出本账期应补收或退款金额。

这个需求隐藏了多个关键规则：账期天数必须来自起止日期，不能固定 30 天；2024 年 2 月是 29 天，2026 年 2 月是 28 天；优惠券先于税；税作用于净差额；只在最终金额做 round；降级退款保留负数；非法日期拒绝。Direct baseline 的主要失败正是把账期固定为 30 天，导致 2026 年 2 月、2024 闰年 2 月和 2026 年 1 月退款 case 出错。

Vibe2Spec 组的差异来自 artifact 链路：需求阶段将“按天计费”拆成 EARS 规则；验收阶段把 28 天、29 天、31 天退款和非法日期变成 fixture；设计阶段记录 UTC date-only arithmetic 和退款符号保留；实现 prompt 要求遵循 Spec / ADR / Acceptance Criteria；验证阶段直接调用核心计算函数，而不是只看页面是否渲染。该实验说明，边界规则需要尽早进入验收材料。

6 验证与质量保证

仓库级验证包括单元测试、静态检查、类型检查和报告构建。当前测试覆盖 CLI 入口、配置解析、prompt 渲染、执行流程、用户追问、缺失产物处理、artifact 校验、Python bootstrap 和默认配

Table 5 Proration 关键失败 case

Case	Direct	Vibe2Spec / 期望	说明
2026 年 2 月，10% coupon，8.25% tax	27.28	29.23	2026 年 2 月实际账期为 28 天。
2024 闰年 2 月	30.00	31.03	2024 年 2 月实际账期为 29 天。
2026 年 1 月降级退款	-32.00	-30.97	1 月账期为 31 天，退款保留负数。

Table 6 Proration 实验中的 artifact 作用

环节	处理的问题	产生的效果
需求阶段	实际账期天数、coupon-before-tax、tax on net delta、final-only rounding、refund 负数、非法日期	模糊“按天计费”被拆成可审阅的 EARS 规则
验收阶段	28 天 2 月、29 天闰年、31 天退款、非法日期	边界条件变成具体数值 fixture，如 29.23、31.03、-30.97
设计阶段	JavaScript 日期解析受时区或 DST 影响、退款符号保留	采用 UTC date-only day difference，并保留负数 refund
实现阶段	编码阶段不重新自由解释需求	要求遵循 Spec / ADR / Acceptance Criteria
验证阶段	页面可渲染不等于计费正确	直接调用核心计算函数验证 6 个探针

置同步。新增回归测试要求：当 proposal 只包含旧版 Context、Goals、EARS、ADR、Acceptance Criteria 和 Out of Scope，但缺少质量镜头时，validate-artifacts 必须失败；普通日志中内联提到 HARNESS_NEEDS_USER_INPUT 或出现“请确认”等词时，不能误触发暂停。

现有 32 个单元测试覆盖了轻量 skill flow 初始化、artifact 内容校验、默认 start 流程、run id 唯一性、prompt style 注入、上下文输入存在与缺失记录、非标准阶段序列拒绝、嵌套配置字段类型校验、execute 模式下真实产物校验、用户补答后同阶段恢复执行、缺失产物重试/跳过/停止、命令失败状态记录、Python bootstrap 配置生成、包内默认配置与示例配置一致、清理旧 run 目录以及 prompt 通过 stdin 传递给执行命令。

Table 7 当前验证闭环

类型	命令或证据	结果
单元测试	<code>PYTHONPATH=src python3 -m unittest discover -s tests</code>	32 个测试通过。
静态检查	<code>.venv/bin/python -m ruff check .</code>	通过。
类型检查	<code>PYTHONPATH=src .venv/bin/python -m mypy src tests</code>	通过。
报告构建	项目内 TinyTeX + xelatex	本报告可编译为 PDF。

7 讨论、局限与后续工作

是否使用 Vibe2Spec 应按任务风险决定。Direct Codex 适合需求清楚、影响范围小、反馈快的小任务；Vibe2Spec 适合业务规则隐含、后续要维护、需要复盘或多人审阅的任务。边界条件如果只停留在一句自然语言里，模型可能按常见经验实现；写成验收 fixture 后，错误更容易在实现前或验证时暴露。

与 OpenSpec 类方法相比，本项目更贴近 Codex 本地开发和课程展示：它直接围绕 Codex CLI、Codex skill 和本地文件系统设计；日常默认使用 vibe2spec-flow，不强制完整平台化流程；最终产物不是停在 spec，而是生成可执行的 Codex 实现指令。代价是当前规范能力、生态成熟度和通用协作能力仍弱于大型开源规范框架。

当前项目仍有明确局限。首先，实验样本少，Direct baseline 和 Vibe2Spec 组不是多次随机采样平均值，不能得出统计意义上的通用结论。其次，artifact 内容校验仍是关键词和概念组级别的检查，不能证明文档内容真的完整、正确或可执行。第三，conversation 隔离主要体现在 prompt 和目录结构上，仍依赖 Codex CLI 的真实会话行为。第四，resume 已有 run 的能力还不完整，安全边界也仍较粗；harness 负责阶段约束，不负责细粒度文件写入审批。

7.1 后续工作

后续优先级包括：

1. 增加 `resume --run-dir`，支持中断后从已有 run 继续；
2. 增强 artifact 内容校验，例如结构化 schema、验收 fixture 解析和 ADR 完整性检查；
3. 为 vibe2spec-flow 增加同步测试，避免 skill 模板和默认 harness 配置漂移；
4. 设计更严格的 direct vs Vibe2Spec 多样本实验，统计首轮通过率、漏需求数量、返工轮数和耗时；
5. 将 proration case 扩展为课程展示脚本，包括输入、运行命令、截图、probe 结果和讲解稿；
6. 如果未来重新引入 subagents，需要增加写域 ownership、合并协议、冲突检测和失败重试。

8 结论

Vibe2Spec 是一个面向 Codex 的 Spec-first 流程原型。它通过固定四阶段 artifact、上下文隔离、运行审计、显式交互协议和内容校验，把一次开发请求拆成可审阅、可验证、可复盘的文件链路。

实验结果显示：简单任务上 Direct Codex 与 Vibe2Spec 可以打平；复杂 UI 任务上，Vibe2Spec 留下了更完整的过程文件；隐含业务规则任务上，Vibe2Spec 把关键边界提前写成验收 fixture，减少了账期计算错误。项目重点不在代码规模，而在于把需求、设计、验证和复盘环节落实到文件、命令和测试中。

参考资料

- [1] OpenAI. Codex and Codex CLI documentation. 本项目围绕本地 Codex CLI 工作流实现。
- [2] Mavin, A.; Wilkinson, P.; Harwood, A.; and Novak, M. 2009. Easy Approach to Requirements Syntax (EARS). 需求阶段采用 EARS 风格组织功能需求。
- [3] Nygard, M. Architecture Decision Records. 本项目使用 ADR 思路记录关键设计决策、原因和替代方案。
- [4] Spec-first software engineering practice. 本项目将需求、设计、任务和实现 prompt 文件化，用于降低上下文漂移和验证不足风险。